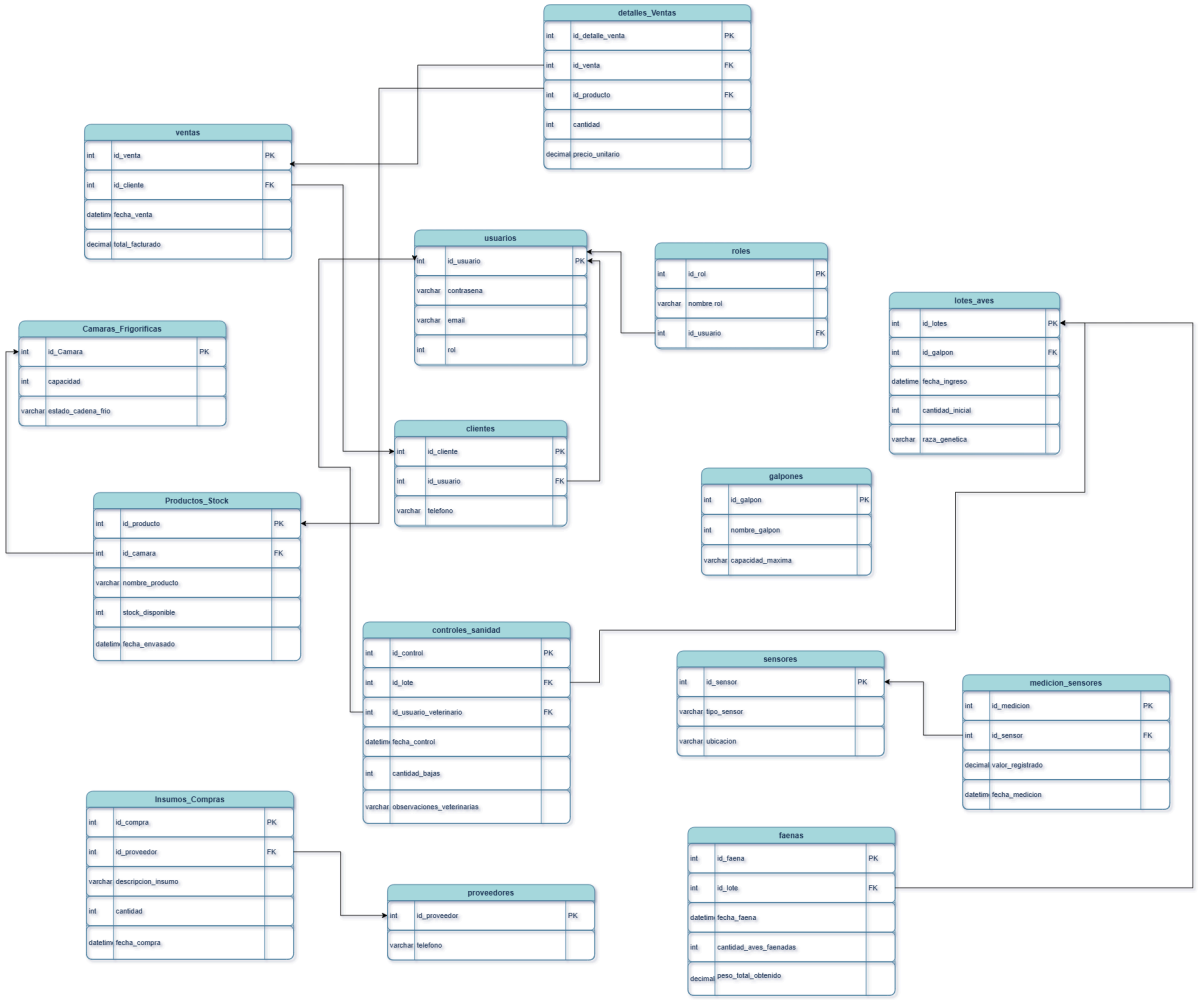
30//8/2026

Comencé con la elaboración del Diagrama Entidad-Relación (DER) borrador del proyecto para estructurar conceptualmente las entidades de la productora avícola, con el objetivo de continuar posteriormente con el Modelo Entidad-Relación (MER) y definir las tablas iniciales.

31/8/2026

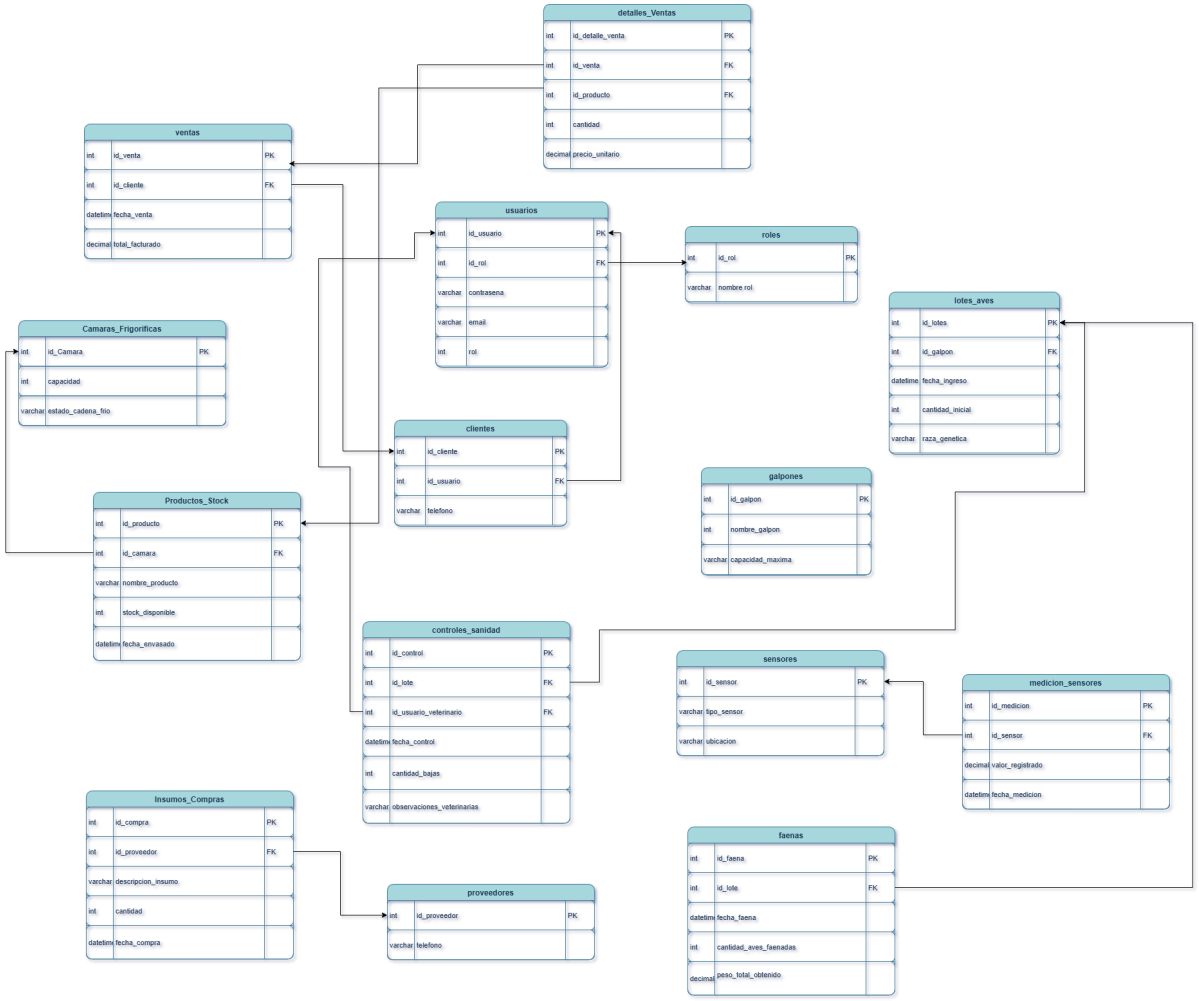
Logré finalizar el primer boceto completo del DER, definiendo las entidades principales (usuarios, roles, clientes, ventas, detalles_ventas, productos_stock, cámaras frigoríficas, galpones, lotes_aves, controles_sanidad, sensores, mediciones, faenas, insumos y proveedores) y trazando sus relaciones preliminares.



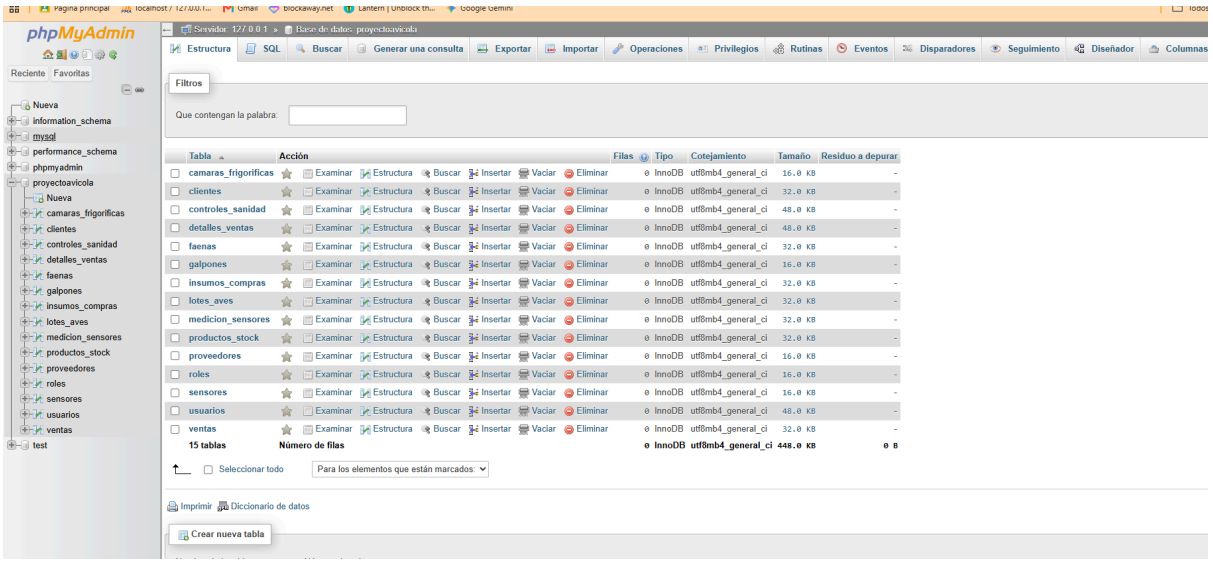
Tuvimos una reunión clave con el profesor de Redes, quien nos convocó para orientarnos en la planificación de la infraestructura técnica. Nos instó a definir la cantidad total de dispositivos que requerirá la productora avícola a partir de los requerimientos calculados en la plataforma de proyectos. Con esa cifra base, deberemos determinar la distribución adecuada de los nodos en las distintas sucursales. Nos aconsejó seleccionar una topología y dimensionamiento acordes a las necesidades operativas para terminar de detallarlo mañana en la reunión con el cliente.

01/09/2026

Corregí una inconsistencia en el diseño del DER en la relación entre USUARIOS y ROLES. Anteriormente, la clave foránea id_usuario estaba ubicada incorrectamente en la tabla ROLES; se reestructuró la asignación para colocar id_rol dentro de la tabla USUARIOS, respetando la cardinalidad adecuada (un rol puede pertenecer a varios usuarios).

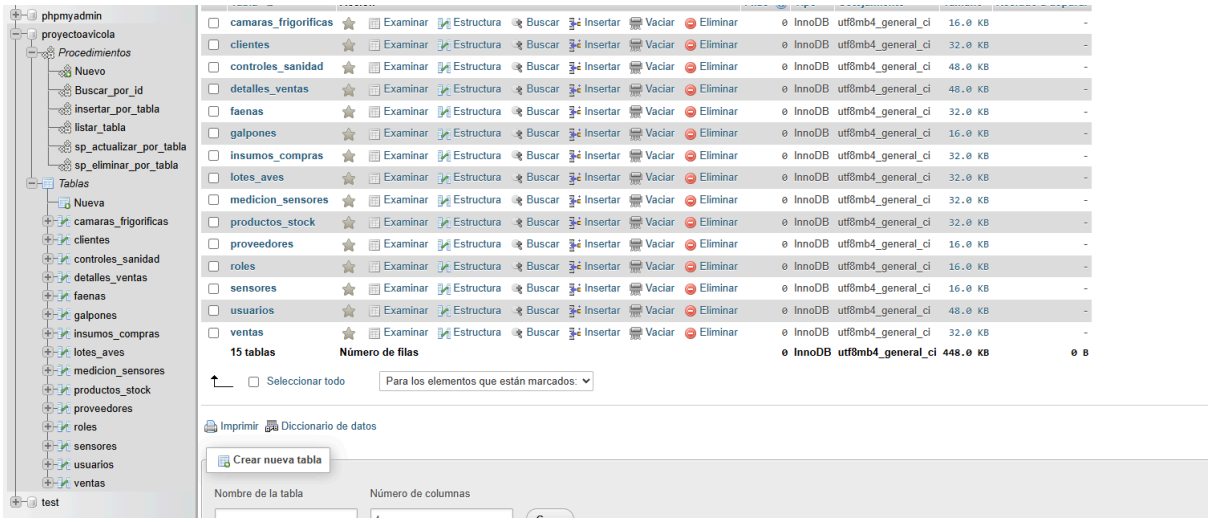


Con el DER corregido y validado, procedí a implementar el diseño lógico y físico de la Base de Datos en phpMyAdmin (proyectoavicola), creando la totalidad de las 15 tablas necesarias con sus respectivas claves primarias, foráneas y tipos de datos correspondientes.



14/ 9

Reestructuré la lógica de la base de datos optimizando los procedimientos almacenados. Se eliminaron los 75 procedimientos individuales redundantes que se habían generado por tabla y se unificaron en 5 stored procedures multipropósito con responsabilidades bien definidas (Buscar_por_id, insertar_por_tabla, listar_tabla, sp_actualizar_por_tabla y sp_eliminar_por_tabla).



22/9

Nos reunimos con el profesor de Programación Orientada a Objetos (POO) para revisar la gestión de operaciones del sistema. Nos dio indicaciones específicas para reforzar el modelo de datos y la lógica en las secciones críticas del negocio, haciendo especial énfasis en garantizar la consistencia en el registro de ventas y el control estricto de stock de productos.

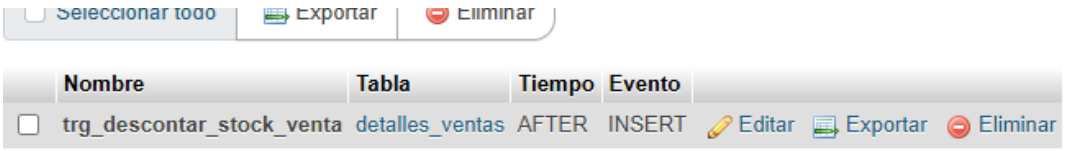
5/10

Hoy lunes el profe de POO nos llamó para orientarnos un poco con la sección de su proyecto, en resumidas cuentas para sugerirnos que revisemos las normas de negocio, sintaxis y modelo de nuestro proyecto.

16/10

Se completó y entregó la documentación técnica requerida para la materia Redes, compuesta por un total de 20 planos detallados que contemplan la distribución física y lógica de los dispositivos, el cableado estructurado y la ubicación de los equipos en las instalaciones de la productora.

19/10
Se realizó la entrega formal del documento de la “capa de negocio” solicitado en POO, donde se detalla la lógica operativa y las reglas del sistema. En el aspecto técnico de la base de datos, se programó e implementó un trigger automatizado que descuenta la cantidad de stock disponible en la tabla productos_stock de forma transparente cada vez que se inserta un nuevo registro en detalles_ventas.



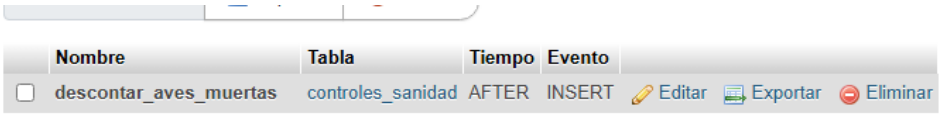
26/10

Se desarrolló el procedimiento almacenado procesar_faena_lote bajo el motor MariaDB. El objetivo del día fue asegurar la consistencia de los datos al momento de enviar aves a faena. Se estructuró un bloque de código atómico (START TRANSACTION / COMMIT) que registra la producción de carne en la tabla faenas y, simultáneamente, descuenta de forma automática las aves procesadas de la tabla lotes_aves. Además, se incorporó un manejador de excepciones (DECLARE EXIT HANDLER FOR SQLEXCEPTION) con funciones de ROLLBACK para revertir cualquier operación en caso de fallos del sistema, evitando descuadres de inventario en los galpones.



9/11
Según el Caso de Uso **CU-14 (Cargar informes sanitarios)**, cada vez que un veterinario inserta un nuevo informe en la tabla `controles_sanidad`, el sistema debe descontar automáticamente las aves muertas (`cantidad_bajas`) de la columna

`cantidad_inicial` de ese lote en la tabla `lotes_aves`. Para automatizar esto, creé el trigger `descontar_aves_muertas` (`AFTER INSERT` sobre `controles_sanidad`).



10/11

Siguiendo la excepción `E1` del **CU-14 (Cargar informes sanitarios)**, la cantidad de bajas registradas en un control sanitario no puede superar la cantidad de aves que tiene actualmente el lote (`cantidad_inicial` en la tabla `lotes_aves`). Para asegurar esta validación antes de insertar el registro en `controles_sanidad`, creé el procedimiento almacenado `registrar_baja_sanitaria` utilizando una variable local y un condicional `IF` con `SIGNAL SQLSTATE '45000'`.

